



Summary of slides

1.05.01 :

- Conceptual Modeling and UML Basics : Conceptual modelling is an essential process used to capture ideas, visualize real-world systems, and create softwer construction plans. It can be applied at any stage of development, starting from the very beginning. To visualize these concepts, the UML is commonly used. UML utilizes use case diagrams to define user goals, class diagrams to structure systen objects, interaction diagrams for communication, and state charts for internal behaviors

- Softwere Architecture and Design Principle: Building robust system requires organizing softwere into loosely coupled layers (such as User Interface, Application Logic and Operation System), where each layer provides a well-defined abstraction to those above it. To further reduce accidental complexity and unnecessary dependencies,

DD MM AA

developers should always program to an interface or abstraction. Additionally, when designing class relationships, aggregation is preferred over inheritance because it can be modified dynamically at runtime, offering much greater flexibility.

- Clean Coding Practices and Testing

At the code level, encapsulation must be maintained by keeping class variables private, as public access directly violates object-oriented principles. Code readability and maintainability depend heavily on meaningful identifiers: class names should be nouns in PascalCasing, variables should use lowerCamelCasing, and methods must be action verbs. Finally, individual units of code must be validated using executable unit test cases to uncover design errors, verify correctness, and enable efficient regression testing.

1.05.07:

• UML (Unified Modeling Language) is like a blueprint language for software. It helps developers visualize, build, and document software system before writing the actual code. Think of it like drawing plans before building a house. It uses different diagrams to show different parts of the system, acting like sentences that make up a whole book (the complete model) it is flexible and can be used in almost any software project

• The Different Views of a System: A complex system can't be understood all at once, so we look at it from different "views" just like reading different chapters in a book. Each view focuses on a specific part of the system. Three main ways to look at a system are:

- Analysis Perspective: Focuses on understanding the real-world problem as it is.
- Specification Perspective: Focuses on designing a solution and showing what the software needs to become.
- Implementation Perspective: Focuses on the actual software parts that will be built

• Types of UML Diagrams: UML uses two main categories of diagrams to show how a system works:

- Structural Diagrams: These show the static parts of the system (how it's built). For example. Class Diagrams show the different pieces of the system (classes) and how they connect. Component and Deployment Diagrams show the software files and the hardware they run on
- Behavioral Digrams: These show how the system acts and moves. For example. Use Case Diagrams show what the users want to do with the system. Sequence and Activity Diagrams show the spstep-by-step flow of actions over time.

• Tpes for Making Good Diagrams

The most important rule is that diagrams are for communication, not just for making pretty pictures. Keep them organized by giving them clear names and arranging the pieces so they don't look messy with too many crossing lines. Don't make them overly complicated, but don't oversimplify them either. Group related things together to make them easier to read, and let software tools help you draw them.

DD MM AA

1.05.08:

The main pieces: Actors and Use Case

Actors are the people, other programs or hardware devices that interact with our softwer from the outside. They always have a specific goal and are usually drawn as stick figures. On the other hand, use cases represent what the user actually wants to achiere with the system. They focus entirely on what the system needs to do and why, rather than worrying about how the code will actually work behind the scenes.

How things connect (Relationships): Elements in a diagram conned in dofferent ways. For example, an actor can be a specific version of a general one, like a "Graduate Student" being a specific type of the general "Student" actor. Use cases also connect to each other in three main ways: an "Extend" relationship acts like an optional add-on that adds extra steps under certains conditions. An "Include" relationship is a required step, meaning one use case absolutely must use another to finish its job. Finally "Generalization" happens when one use case is simply a more specific version of a broader one.

- A use case diagram gives us a helpful visual map made up of the users (Actors) their goals (Uses Cases) how they connect (Relationships), and the system itself. These diagrams are super useful because they capture exactly what the user will see, help us plan our project schedule, and give us a clear guide for testing the software later on

1.05.09:

The Building Blocks - Objects and Classes: Objects are the fundamental building blocks of a system; they represent any "thing" with spacetime boundaries, identity, and encapsulated state or behavior. Because systems contain many objects, humans naturally classify similar things together into classes. A class is an abstraction that allows us to manage whole collections of objects sharing common characteristics. Visually, a class is draw as a box divided into compartments for its name, attributes, and operations. Names should be singular nouns in Upper Camel Case (ex. Student). Attributes (properties holding values) and operations (the methods or actions) use lower Camel Case and include symbols like + for public, - for private, and # for protected visibility

Relationships: How Classes Connect: Classes dont work alone, they connect in several ways, Generalization occurs when one class is a specific subtype of a more general one (ex. a "Graduate Student" is a subset of "Student"). Associations are structural links indicating that objects of one class are connected to another (ex. Student "is a registered for" a Semester). These associations use "Multiplicity constraints" to define the numerical limits of the connection, like 1..* meaning "one to many". Aggregation and Composition are special "is part of" associations. Aggregation (an empty diamond) means the parts can exist independently, while Composition (a solid black diamond) means the part's life is entirely tied to the whole object. Finally, a Dependency (a dashed arrow) means one class "uses" another; if the used class changes the dependent one might have to change too.

- A class diagramm brings everything together, containing classes, individual objects, and their relationships (associations, generalization, dependencies). You can think of these diagrams

DD MM AA

as the paragraphs of a technical paper, where all diagrams combined create a complete "model" of the system. When making them, it's crucial to maintain a consistent level of abstractions so they aren't confusing, and developers must be careful not to focus so much on structure that they ignore the system's behavior

1.05.10:

- When starting your UML analysis, it helps to think of your entire model as an essay and each diagram as a single paragraph. Your goal is to model the real-word problem as it is, without worrying about the actual software code yet. Drawing these diagrams should naturally help you ask questions and explore how the system works

Use case diagrams: These diagrams focus on who uses the system and what they want to achieve. An "Actor" is someone or something outside the system (like a user) that has a specific goal. The goal itself is called a "Use Case". Sometimes, these goals depend on each other. If a main goal absolutely requires a smaller sub-goal to work, you use an "Include" dependency. On the other hand, if a goal just has an optional variation or an extra step for special situations, you use an "Extend" dependency. You can also show that one actor is a more specific version of a general actor using a "Generalization" Relationship

Class Diagrams: These diagrams map out the real-world thing (objects) inside your system. A "class" groups together objects that share common traits. Classes connect to each other in a few main ways:

- An association is a basic, persistent link between two classes. To define exactly how many objects can be linked together, you add Multiplicity constraints (like 1 to many)
- A generalization shows that one class is a specific subtype of another (for example saying every B object is also an A object)
- When a class is made up of smaller parts, you use Aggregation or Composition. Aggregation is a loose "is part of" connection, while Composition is very strict - if the main object is destroyed, all of its parts are destroyed with it.